

объектно ориентированное программирование

ОСНОВЫ

ВВЕДЕНИЕ

изначальная разработка

зачем нужно проектирование?

- разработка “с чистого листа” – процесс с минимальной сложностью
 - нужно продумать что хотим сделать
 - нужно продумать как хотим сделать
 - МОЖНО МЕНЯТЬ ВСЁ ЧТО ЕСТЬ
 - нет риска сломать существующую логику/данные (их нет)

поддержка

зачем нужно проектирование?

- поддержка добавляет проблем, которых не существовало при изначальной разработки
- потеря контекста функционала
 - документация устаревает
 - память человека не так надёжна
- потеря технического контекста
 - разработка это про компромиссы
 - на реализацию может влиять текущая ситуация, в будущем эта рационализация может испариться – решения станут не понятны

проблематика

зачем нужно проектирование?

- при доработке функционала
 - основная цель – добавить новый функционал
 - главная цель – не сломать существующий

хорошая? архитектура

зачем нужно проектирование?

- цель – не столько упростить корректные доработки, сколько усложнить не корректные
- расширение vs изменение
 - расширение – добавление нового способа делать уже существующее действие
 - изменение – добавление в принципе нового функционала
- контр-интуитивно, но хороший дизайн может усложнять внесение изменений
 - хорошую систему не должно быть просто сломать, она должна сопротивляться

простота ≠ топорность

зачем нужно проектирование?

- расставляйте приоритеты правильно
 - не цель – сделать так чтобы было просто написать
 - цель – сделать так чтобы было просто понять через два года
- реализуя систему, мы выстраиваем в голове какое-то её представление
 - если это представление не отражено в коде – при его чтении другой человек должен будет расшифровать его при чтении кода
 - расшифровать вполне можно не правильно – больше вероятность что-то сломать

простота ≠ топорность

зачем нужно проектирование?

- топорный код
 - не содержит абстракций
 - “не переусложнён”
 - но на поверку – не имеет структуры и никак не помогает разобраться
- простой код
 - может нести за собой дополнительную когнитивную нагрузку
 - но более структурирован: позволяет взглянуть на логику на разных уровнях; отражает намерения разработчика; progressive disclosure контекста системы

ЧТО ЭТО ТАКОЕ?

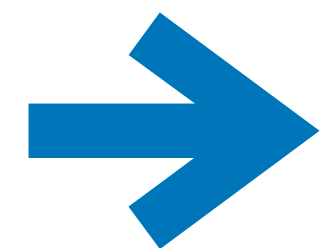
парадигмы программирования

- набор правил для решения определённых проблем
- набор ограничений, отбирающих у нас определённые свободы при выборе решения
 - опять контр-интуитивно – но это хорошо

структурное программирование

парадигмы программирования

```
0  VAR i
1  SET i 1
2  PRINT i
3  INC i
4  JIFLS i 10 2
```



```
for (var i = 1; i < 10; i++)
{
    Console.WriteLine(i);
}
```

процедурное программирование

парадигмы программирования

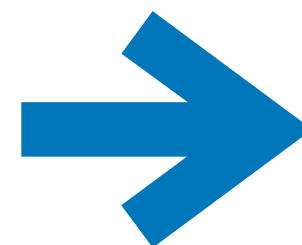
```
var probability1 = Random.Shared.NextDouble();
var coefficient1 = probability1 > 0.5 ? 10d : 0;

var probability2 = Random.Shared.NextDouble();
var coefficient2 = probability2 > 0.5 ? 10d : 0;

var input = Console.ReadLine();
var value = input is null ? 0 : int.Parse(input);

var result = value * coefficient1 / coefficient2;

var message = $"result: {result}";
Console.WriteLine(message);
```



```
var coefficient1 = CalculateCoefficient();
var coefficient2 = CalculateCoefficient();

var value = ReadValue();

var result = value * coefficient1 / coefficient2;
OutputResult(result);

static double CalculateCoefficient()
{
    var probability = Random.Shared.NextDouble();
    return probability > 0.5 ? 10d : 0;
}

static int ReadValue()
{
    var input = Console.ReadLine();
    return input is null ? 0 : int.Parse(input);
}

static void OutputResult(double result)
{
    var message = $"result: {result}";
    Console.WriteLine(message);
}
```


мы научимся

в рамках курса

- объектно-ориентированному проектированию
- C#
- писать объектно-ориентированный код на объектно-ориентированном языке

основные концепции ООП

инкапсуляция

**принцип объединения атрибутов и поведений
(данных и методов) в рамках одного типа**



инкапсуляция

инкапсуляция

основные концепции ООП

- **тип – шаблон**, описывающий какие данные и методы имеет объект
- **объект – экземпляр** типа, существующий во время выполнения кода, занимающий память

инкапсуляция

основные концепции ООП

```
public class SampleClass
{
    public int _firstField;
    public int _secondField;
}
```

- **ссылочные** типы
- данные объекта хранятся **на куче**
- на стеке хранится только ссылка

```
public struct SampleStruct
{
    public int _firstField;
    public int _secondField;
}
```

- **значимые** типы
- данные хранятся там, где находится объект структуры

ИНКАПСУЛЯЦИЯ

ОСНОВНЫЕ КОНЦЕПЦИИ ООП

```
public class BankAccount
{
    public decimal _value;

    public BankAccount(decimal value)
    {
        _value = value;
    }

    public bool TryWithdraw(decimal amount)
    {
        if (amount > _value)
            return false;

        _value -= amount;
        return true;
    }
}
```

инкапсуляция

основные концепции ООП

- забирает свободу размещения логики
 - логика не может жить сама по себе
 - логика должна принадлежать типу

инкапсуляция

основные концепции ООП

- заставляет моделировать логику в виде сущностей
 - приводит к более детальному продумыванию дизайна
 - приводит к более четкому отражению намерения автора кода

инкапсуляция

основные концепции ООП

- заставляет столкнуться с проблемой связей между данными/сущностями/логикой
 - если не продумать связи грамотно – модель превратиться в кашу
 - без выделения объектной модели, связи всё ещё существуют, но они не явные (что хуже)
- локализация логики работающая с одним набором данных

СОСТОЯНИЕ vs параметр

инкапсуляция

процедурное

```
public static double Advance(double velocity, double time)
{
    return velocity * time;
}
```

объектно-ориентированное?

```
public class Vehicle
{
    public static double Advance(double velocity, double time)
    {
        return velocity * time;
    }
}
```

состояние vs параметр

инкапсуляция

- метод в классе – ещё не ООП
- всегда есть вопрос в распределении данных
 - что сделать атрибутом?
 - что сделать параметром?
 - что сделать возвращаемым значением?

состояние vs параметр

инкапсуляция

```
public class Vehicle
{
    public double _velocity;

    public double Advance(double time)
    {
        return _velocity * time;
    }
}
```

состояние vs параметр

инкапсуляция

- ориентируйтесь на предметную область, типы описывают определённые субъекты и их операции
- принадлежность и изменяемость данных
 - что нужно для выполнения всегда?
 - какие данные могут меняться в зависимости от контекста?
- отделяйте состояние от результата
 - то, что значение получено в метода типа, ещё не значит что оно должно быть его атрибутом

**набор правил, определяющих
корректное состояние данных**



инвариант данных

**набор данных, их инварианта и поведений,
позволяющих изменять эти данные согласно их
инварианту**



инвариант типа

основные концепции ООП

сокрытие

СОКРЫТИЕ

ОСНОВНЫЕ КОНЦЕПЦИИ ООП

```
var acc = new BankAccount(0);  
acc._value = 1000;  
  
if (acc.TryWithdraw(100))  
{  
    Console.WriteLine("Вы совершили финансовую махинацию!");  
}
```

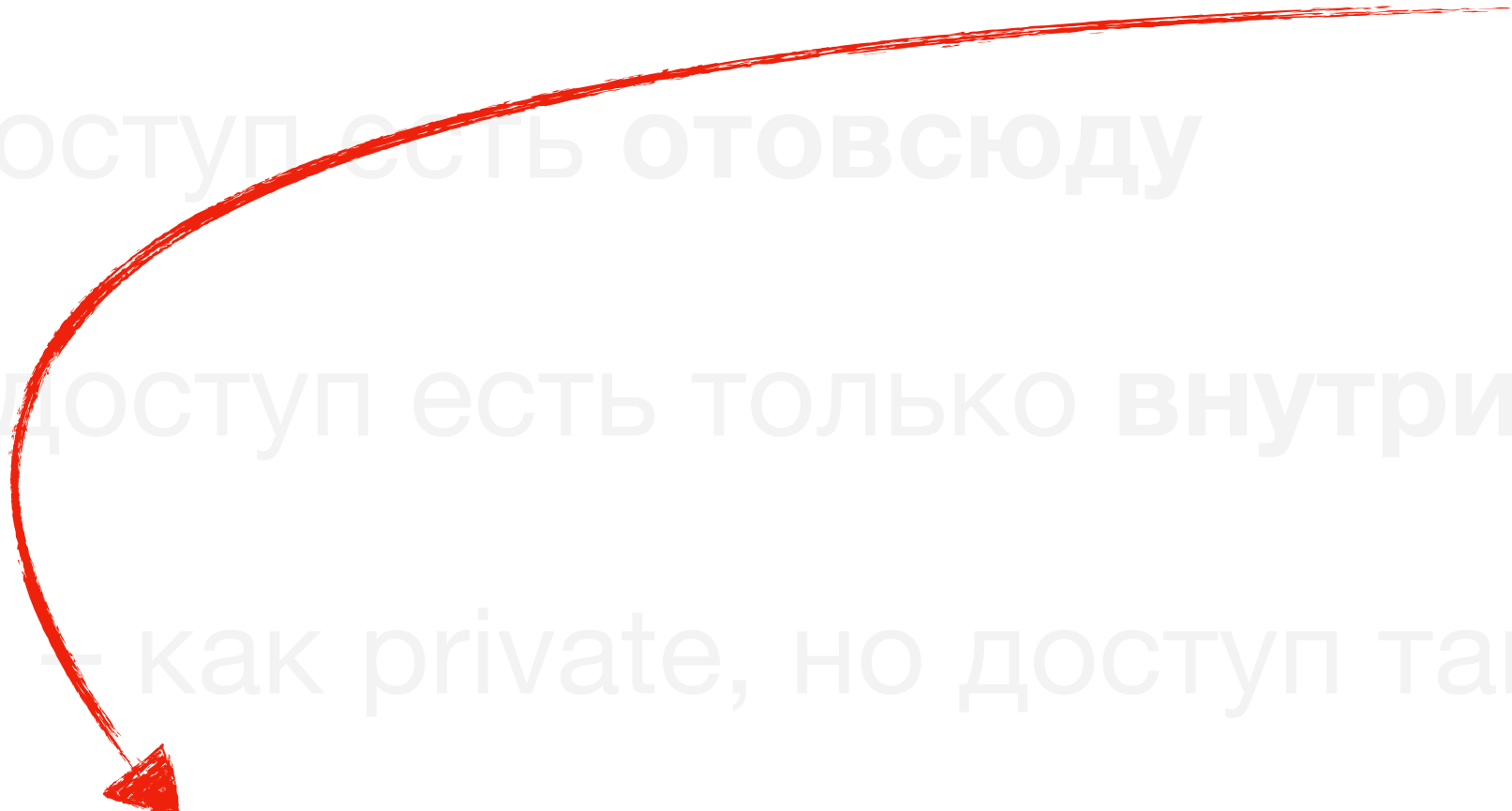
сокрытие

основные концепции ООП

- отбирает свободу доступа к данным/поведениям
- ограничить доступ к данным и методам позволяют модификаторы доступа
 - `public` – доступ есть **отовсюду**
 - `private` – доступ есть только **внутри типа**
 - `protected` – как `private`, но доступ также есть **у дочерних типов**
 - `internal` – доступ есть только в рамках **текущей сборки** (модуля)

сокрытие

основные концепции ООП

- ограничить доступ к данным и методам позиции доступа
 - `public` – доступ есть отовсюду
 - `private` – доступ есть только внутри типа
 - `protected` – как `private`, но доступ также есть у дочерних типов
 - `internal` – доступ есть только в рамках текущей сборки (модуля)
- использование `internal` для обеспечения сокрытия в бизнес логике свидетельствует о не правильно выстроенной абстракции
- 

СОКРЫТИЕ

ОСНОВНЫЕ КОНЦЕПЦИИ ООП

```
public class BankAccount
{
    private decimal _value;

    public BankAccount(decimal value)
    {
        _value = value;
    }

    public bool TryWithdraw(decimal amount)
    {
        if (amount > _value)
            return false;

        _value -= amount;
        return true;
    }
}
```

код не скомпилируется

```
var acc = new BankAccount(0);
acc._value = 1000;
```

инкапсуляция + сокрытие

основные концепции ООП

- объединяя инкапсуляцию и сокрытие мы можем гарантировать соблюдение инварианта типа
- объединяя инкапсуляцию и сокрытие мы получаем **абстракцию**
 - представляет собой “**черный ящик**” с точки зрения реализации
 - позволяет писать **более простой код** на более высоких уровнях абстракции
- хороший дизайн – локальные, самодостаточные абстракции + их комбинация на уровне выше

СВОЙСТВА

```
public class BankAccount
{
    private decimal _value;

    public decimal GetValue() ⇒ _value;

    public bool TryWithdraw(decimal amount)
    {
        if (amount > _value)
            return false;

        _value -= amount;
        return true;
    }
}
```

СВОЙСТВА

- Без свойств
 - нужно писать отдельный метод получения данных
 - нужно содержать отдельное поле
- Со свойствами
 - Не нужно **лишних** методов
 - Описывают **хранимые данные** и **способы работы** с ними
 - Сохраняют “**семантику поля**”

АВТО-СВОЙСТВА

СВОЙСТВА

```
public class BankAccount
{
    public decimal Value { get; private set; }

    public bool TryWithdraw(decimal amount)
    {
        if (amount > Value)
            return false;

        Value -= amount;
        return true;
    }
}
```


ВЫЧИСЛЯЕМЫЕ СВОЙСТВА

СВОЙСТВА

```
public class ShoppingCart
{
    public decimal TotalCost ⇒ Items.Sum(item ⇒ item.Cost);
}
```

СВОЙСТВА VS ПОЛЯ

свойства

- свойства и поля не взаимоисключающие конструкции
 - поля – внутренне состояние, которое мы хотим скрыть
 - свойства – состояние, которое открываем наружу
- внешняя логика может использовать атрибуты сущности
- цель сокрытия определяет выбор конструкции
 - не отдавайте мутабельные коллекции наружу

ОСНОВНЫЕ КОНЦЕПЦИИ ООП

КОМПОЗИЦИЯ

КОМПОЗИЦИЯ

основные концепции ООП

```
public class SampleClass
{
    public int _firstField;
    public int _secondField;
}
```

**способ создания одних типов на основе других,
при помощи хранения объектов одних типов в
объектах других типов**



КОМПОЗИЦИЯ

агрегация

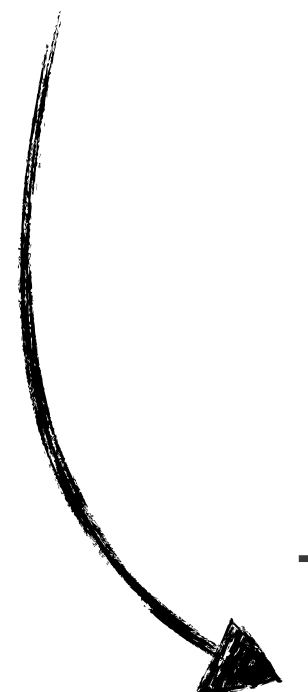
ВИДЫ КОМПОЗИЦИИ

```
public struct Point2D
{
    public Point2D(double x, double y)
    {
        X = x;
        Y = y;
    }

    public double X { get; }

    public double Y { get; }
}
```

```
var point = new Point2D(1, 2);
```



```
}
  "X": 1,
  "Y": 2
}
```

**ВИД КОМПОЗИЦИИ, ПОДРАЗУМЕВАЮЩИЙ, ЧТО
ХРАНИМЫЕ ЗНАЧЕНИЯ ПОЛУЧАЮТСЯ ИЗВНЕ**



агрегация

ассоциация

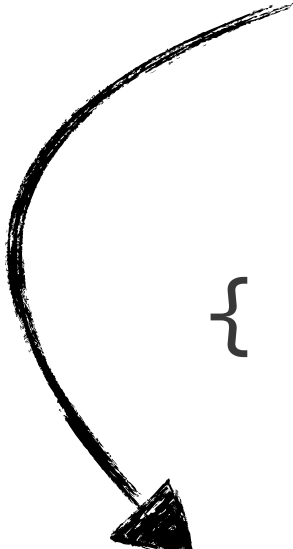
ВИДЫ КОМПОЗИЦИИ

```
public class Car
{
    private readonly Engine _engine;
    private readonly Wheel[] _wheels;

    public Car()
    {
        _engine = new Engine("V8");

        _wheels = Enumerable
            .Range(0, 4)
            .Select(index => new Wheel(index))
            .ToArray();
    }
}
```

```
var car = new Car();
```



```
{
    "_engine": {
        "Name": "V8"
    },
    "_wheels": [
        {"Index": 0},
        {"Index": 1},
        {"Index": 2},
        {"Index": 3}
    ]
}
```


**ВИД КОМПОЗИЦИИ, ПОДРАЗУМЕВАЮЩИЙ, ЧТО
ХРАНИМЫЕ ЗНАЧЕНИЯ СОЗДАЮТСЯ САМИМ ОБЪЕКТОМ**



агрегация vs ассоциация

композиция

- агрегация
 - более низкая связанность между вложенными и содержащими типами
 - мы не определяем как создаются вложенные объекты
- ассоциация
 - более высокая связанность между вложенными и содержащими типами
 - мы сами создаём вложенные объекты

основные концепции ООП

полиморфизм

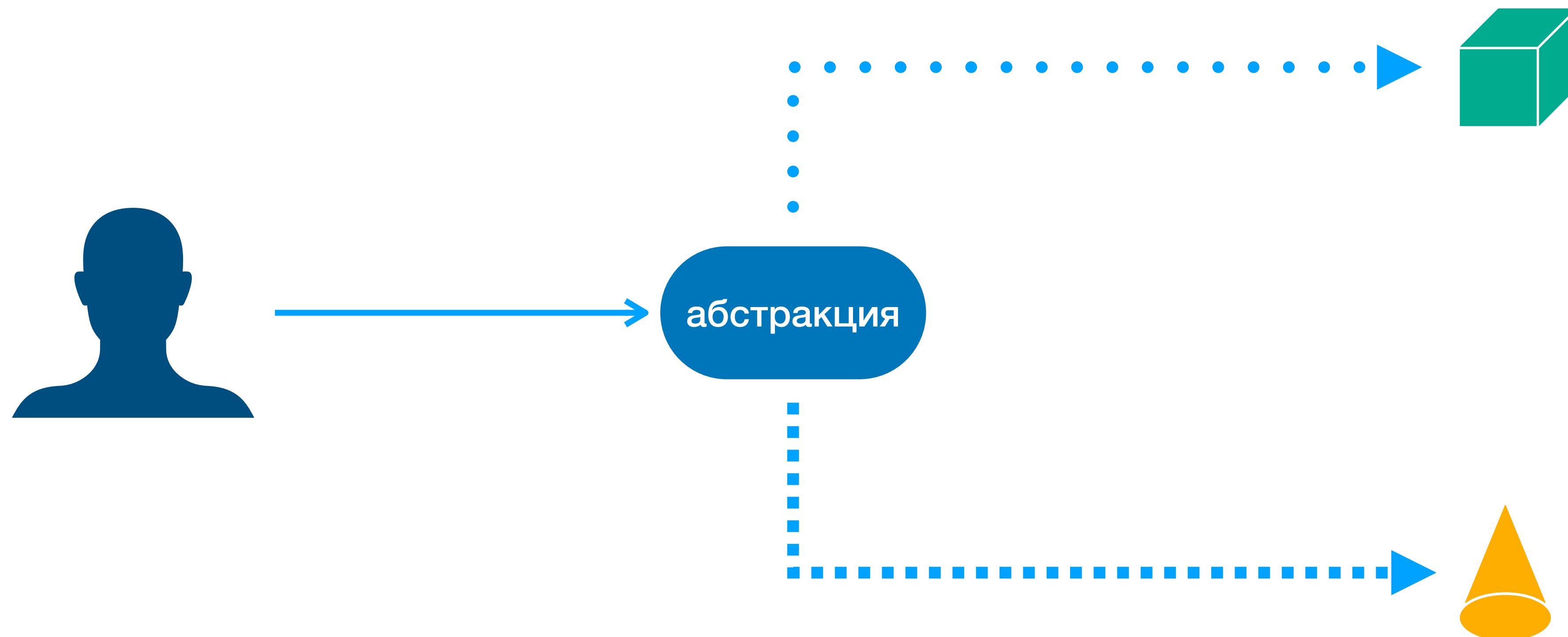
полиморфизм

основные концепции ООП

- подразумевает что логика, написанная один раз, может выполняться по разному
- в контексте ООП – полиморфизм подтипов
- больше отделяем абстракцию от реализации

полиморфизм

основные концепции ООП



интерфейсы

полиморфизм

- определяют только сигнатуры поведений
- не определяют реализаций
- являются контрактом к объекту типа который его реализует
- зачастую называются абстракцией

```
public interface IMovable
{
    Point Location { get; }

    void MoveTo(Point point);
}
```

интерфейсы

полиморфизм

```
public class Car : IMovable
{
    public Point Location { get; private set; }

    public void MoveTo(Point point)
    {
        Console.WriteLine("Wroom-wroom!");
        Location = point;
    }
}
```

```
public struct Stone : IMovable
{
    public Point Location { get; private set; }

    public void MoveTo(Point point)
    {
        Console.WriteLine("Flop-flop");
        Location = point;
    }
}
```

полиморфизм

основные концепции ООП

- интерфейсы позволяют лучше структурировать вариативность логики
- полиморфизм позволяет избежать излишней условной логики
- полиморфизм позволяет моделировать вариативность логики представлением субъектов предметной области в виде типов, реализующих соответствующие им поведения
- отнимает свободу реализации вариативности логики

наследование

полиморфизм

- наследование в C# поддерживают только классы
- в отличие от интерфейсов получаем и реализации базового класса
- дочерние классы могут переопределять реализации родительских классов

Виртуальные методы

наследование

```
public class Car
{
    public Point Location { get; protected set; }

    public virtual void MoveTo(Point point)
    {
        Console.WriteLine("Wroom-wroom!");
        Location = point;
    }
}
```

```
public class FastCar : Car
{
    public override void MoveTo(Point point)
    {
        Console.WriteLine("Wroom-wroom (fast)");
        Location = point;
    }
}
```

абстрактные методы

наследование

```
public abstract class CarBase
{
    public Point Location { get; protected set; }

    public abstract void MoveTo(Point point);
}
```

```
public class FastCar : CarBase
{
    public override void MoveTo(Point point)
    {
        Console.WriteLine("Wroom-wroom (fast)");
        Location = point;
    }
}
```


**отделение абстракции от реализации,
позволяющее прозрачно использовать различные
реализации, посредством единого контракта**



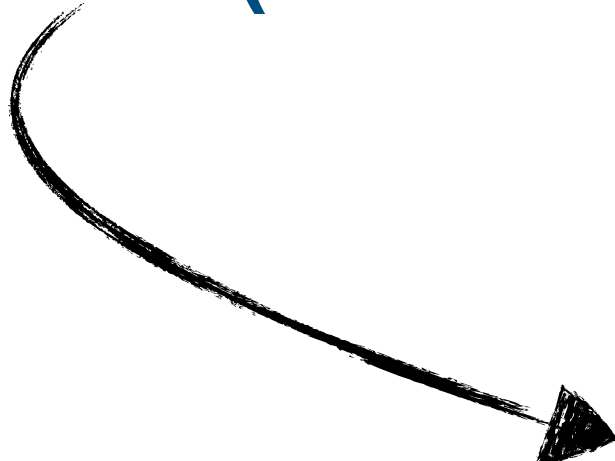
полиморфизм подтипов

используются интерфейсы, в C# реализовывать интерфейсы
могут как классы, так и структуры
говорят, что тип реализует интерфейс
(класс Car реализует интерфейс IMovable)



реализация (наследование поведения)

используются базовые классы,
в С# одна структура не может быть унаследована от другой, либо от класса
говорят, что класс является наследником другого класса, либо же его
подклассом
(класс FastCar является наследником класса Car)



наследование (наследование реализаций)

наследование

полиморфизм

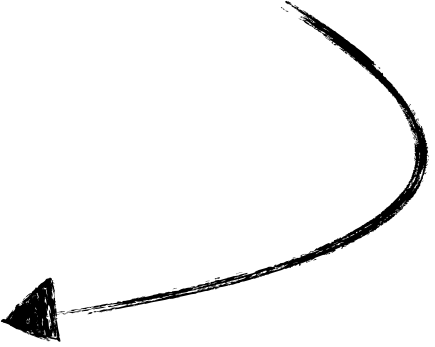
- использовать для реализации полиморфизма
- не использовать для переиспользования бизнес логики
- наследование приводит к сильной связанности между типами
 - со временем становится сложнее разорвать эту связь
 - код становится сложнее рефакторить

наследование

полиморфизм

применение наследования
для переиспользования
данных

ПЛОХО



```
public enum Suit
{
    Hearts,
    Diamonds,
    Clubs,
    Spades,
}

public enum CardValue
{
    Six,
    Seven,
    Eight,
    Nine,
    Ten,
    Jack,
    Queen,
    King,
    Ace,
}

public class Card
{
    public Card(Suit suit, CardValue value)
    {
        Suit = suit;
        Value = value;
    }

    public Suit Suit { get; }

    public CardValue Value { get; }
}
```

```
public class Card
{
    public Card(Suit suit, CardValue value) { ... }

    public Suit Suit { get; }

    public CardValue Value { get; }
}

public class Deck
{
    public Deck(ICollection<Card> cards) { ... }

    public ICollection<Card> Cards { get; }

    public void Shuffle()
    {
        ...
    }
}

public class Dealer : Deck
{
    public Dealer(ICollection<Card> cards) : base(cards) { }

    public void StartGame()
    {
        ...

        Shuffle();

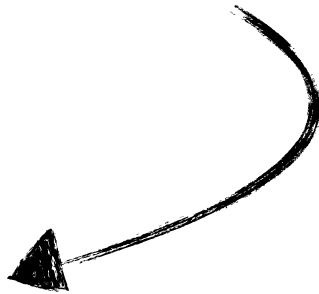
        ...
    }
}
```


наследование

полиморфизм

для переиспользования
логики – используем
композицию

ХОРОШО



```
public class Dealer
{
    private readonly Deck _deck;

    public Dealer(Deck deck)
    {
        _deck = deck;
    }

    public void StartGame()
    {
        ...

        _deck.Shuffle();

        ...
    }
}
```

**совокупность атрибутов и поведений, реализация
и данные которого скрыты от его конечного
пользователя**



объект (с точки зрения теории ООП)

проектирование
объектной модели
имутабельность

различия парадигм

имутабельность

object oriented

```
class Model
{
    public readonly int Value;

    public Model(int value)
    {
        Value = value;
    }
}
```

≠

functional

```
type Model(value: int) =
    let mutable Value = value
```

свойство данных, не подразумевающее изменения

в ООП, используется в виде сокрытия мутабельных данных и имутабельность значений не требующих изменения



имутабельность

ИЗЛИШНЯЯ ИМУТАБЕЛЬНОСТЬ

имутабельность

```
public class StudentGroup
{
    public long Id { get; set; }

    public string Name { get; set; }

    public List<long> StudentIds { get; set; }

    public void AddStudent(long studentId)
    {
        if (StudentIds.Contains(studentId) is false)
            StudentIds.Add(studentId);
    }
}
```

МИНИМИЗАЦИЯ МУТАБЕЛЬНОСТИ

имутабельность

```
public class StudentGroup
{
    private readonly HashSet<long> _studentsIds;

    public StudentGroup(long id, string name)
    {
        Id = id;
        Name = name;

        _studentsIds = new HashSet<long>();
    }

    public long Id { get; }

    public string Name { get; set; }

    public IReadOnlyCollection<long> StudentIds => _studentsIds;

    public void AddStudent(long studentId)
    {
        _studentsIds.Add(studentId);
    }
}
```

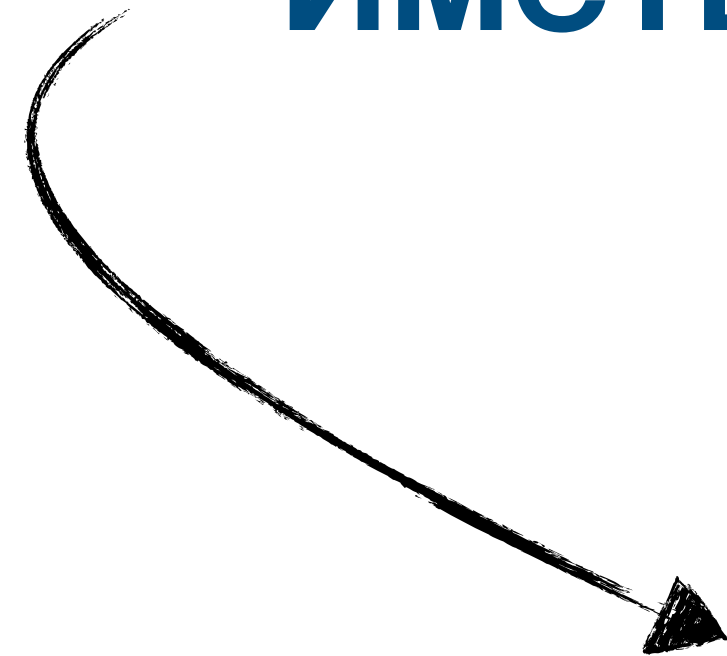
проектирование
объектной модели
обработка ошибок

проблематика

обработка ошибок

- код, обрабатывающий только позитивные сценарии – бесполезен
- способов обрабатывать ошибки много
 - bool флаги
 - исключения

**абстракция, для работы с которой, необходимо
иметь знание о деталях её реализации**



протёкшая абстракция

ИСКЛЮЧЕНИЯ

обработка ошибок

- исключения не отражены в сигнатуре
- поиск конкретного типа исключения и ситуации когда оно кидается приводит к протёкшей абстракции
 - всё становится хуже при работе с интерфейсами
- неудачное выполнение операции $\neq$ исключительная ситуация

result types

обработка ошибок

- выражаем результат операции как объект
 - тип объекта учитывает как успех, так и негативные исходы
- основывается на концепции суммы типов/union-типов
 - заимствование из функционального программирования
- в C# нет union-типов на уровне языка (пока что, уже очень-очень скоро)
 - используем ОО-механики
 - используем закрытые иерархии, где каждый тип – отдельный исход

result types

обработка ошибок

```
public abstract record SomeOperationResult
{
    public record Sucess : SomeOperationResult;

    public record Failure(string ErrorMessage) : SomeOperationResult;
}
```


result types

обработка ошибок

- возвращаемый тип – часть сигнатуры
 - не нужно лезть в детали чтобы узнать какие исходы могут быть
- усиление контракта интерфейса
 - зашиваем поддерживаемые для расширения исходы операции
- добавление новых исходов – возможно, но оно сложнее
 - если внесение ломающих изменений сложнее – меньше вероятность их появления

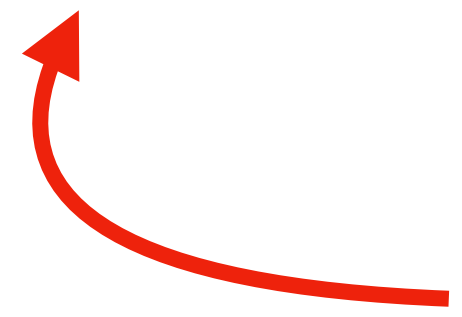
result types

обработка ошибок

```
public abstract record SomeOperationResult
{
    public record Sucess : SomeOperationResult;

    public record Failure(string ErrorMessage) : SomeOperationResult;
}

public record ExternalResult : SomeOperationResult;
```



ПЛОХО

result types

обработка ошибок

```
public abstract record SomeOperationResult
{
    private SomeOperationResult() { }

    public record Sucess : SomeOperationResult;

    public record Failure(string ErrorMessage) : SomeOperationResult;
}
```


result types

обработка ошибок

```
public abstract record SomeOperationResult
{
    private SomeOperationResult() { }

    public record Sucess : SomeOperationResult;

    public record Failure(string ErrorMessage) : SomeOperationResult;
}

public record ExternalResult : SomeOperationResult.Sucess;
```



ПЛОХО

result types

обработка ошибок

```
public abstract record SomeOperationResult
{
    private SomeOperationResult() { }

    public sealed record Success : SomeOperationResult;

    public sealed record Failure(string ErrorMessage) : SomeOperationResult;
}
```

обработка результатов

обработка ошибок

```
public abstract record PlaceOrderResult
{
    private PlaceOrderResult() { }

    public sealed record Success(IReadOnlyList<OrderItem> OrderItems) : PlaceOrderResult;

    public sealed record InsufficientFunds : PlaceOrderResult;

    public sealed record OutOfStock(IReadOnlyList<OrderItem> OrderItems) : PlaceOrderResult;
}
```

```
public interface IShopService
{
    PlaceOrderResult PlaceOrder(User user, Order order);
}
```


обработка результатов

обработка ошибок

```
public ProcessOrderResult ProcessOrder(User user, Order order)
{
    var placeResult = _shopService.PlaceOrder(user, order);

    if (placeResult is PlaceOrderResult.Success success)
        return new ProcessOrderResult.Success(success.OrderItems);

    if (placeResult is PlaceOrderResult.InsufficientFunds)
        return new ProcessOrderResult.Failure("User has insufficient funds");

    if (placeResult is PlaceOrderResult.OutOfStock outOfStock)
    {
        order.ExcludeItems(outOfStock.OrderItems);
        return ProcessOrder(user, order);
    }

    throw new UnreachableException();
}
```

анти-паттерны результатов

обработка ошибок

- избыточность кейсов результата
 - кейсы результата должны выделяться под потенциальные точки вариативности логики, а не под все места в коде где возвращается ошибка
 - пример избыточности – валидация сущности + кейс на ошибку валидации для каждого из проверяемых атрибутов
 - решение – кейс для ошибки валидации + представление ошибки как атрибут кейса (строка или error-object)
 - error object – объект представляющий ошибку, умеет форматироваться в текстовое представление

анти-паттерны результатов

обработка ошибок

- агрегация кейсов результата
 - метод может вызывать несколько операций, возвращающих разный результат
 - если результат метода содержит сумму результатов вызываемых методов – абстракция протекла, сильна связанность

обработка результатов

обработка ошибок

```
public ProcessOrderResult ProcessOrder(User user, Order order)
{
    var placeResult = _shopService.PlaceOrder(user, order);

    if (placeResult is PlaceOrderResult.Success success)
        return new ProcessOrderResult.Success(success.OrderItems);

    if (placeResult is PlaceOrderResult.InsufficientFunds)
        return new ProcessOrderResult.Failure("User has insufficient funds");

    if (placeResult is PlaceOrderResult.OutOfStock outOfStock)
    {
        order.ExcludeItems(outOfStock.OrderItems);
        return ProcessOrder(user, order);
    }

    throw new UnreachableException();
}
```

обработка результатов

обработка ошибок

```
public ProcessOrderResult ProcessOrder(User user, Order order)
{
    var placeResult = _shopService.PlaceOrder(user, order);

    if (placeResult is PlaceOrderResult.Success success)
        return new ProcessOrderResult.Success(success.OrderItems);

    if (placeResult is PlaceOrderResult.InsufficientFunds)
        return new ProcessOrderResult.Failure("User has insufficient funds");

    if (placeResult is PlaceOrderResult.OutOfStock outOfStock)
    {
        order.ExcludeItems(outOfStock.OrderItems);
        return ProcessOrder(user, order);
    }

    throw new UnreachableException();
}
```


анти-паттерны результатов

обработка ошибок

- агрегация кейсов результата
 - метод может вызывать несколько операций, возвращающих разный результат
 - если результат метода содержит сумму результатов вызываемых методов – абстракция протекла, сильна связанность
 - ещё хуже, когда результат собирает все кейсы всех реализаций интерфейсов – повод пересмотреть абстракцию

проектирование объектной модели value objects

проблематика

value objects

- primitive obsession – ситуация когда доменные примитивы выражаются примитивами языка
- доменные примитивы могут иметь более узкую область допустимых значений в сравнении с примитивами языка
- результат – дублирующиеся валидации

проблематика

value objects

```
public class Account
{
    public decimal Balance { get; private set; }

    public WithdrawResult Withdraw(decimal value)
    {
        if (value < 0)
            throw new ArgumentException("Value cannot be negative", nameof(value));

        if (Balance < value)
            return new WithdrawResult.Failure("Insufficiend funds");

        Balance -= value;
        return new WithdrawResult.Success();
    }
}
```

решение

value objects

- выделение абстракции для доменных примитивов – value object'ы
 - позволяют инкапсулировать валидацию в отдельный тип
 - при использовании в коде – знаем что валидация пройдена
 - если бы она не прошла – объект бы не создался

реализация

value objects

```
public struct Money
{
    public Money(decimal value)
    {
        ArgumentOutOfRangeException.ThrowIfNegative(value);
        Value = value;
    }

    public decimal Value { get; }

    public static Money operator -(Money left, Money right)
    {
        var value = left.Value - right.Value;
        return new Money(value);
    }

    public static bool operator <(Money left, Money right)
        ⇒ left.Value < right.Value;
}
```

```
public class Account
{
    public Money Balance { get; private set; }

    public WithdrawResult Withdraw(Money value)
    {
        if (Balance < value)
        {
            return new WithdrawResult.Failure(
                "Insufficiend funds");
        }

        Balance -= value;
        return new WithdrawResult.Success();
    }
}
```

ИСКЛЮЧЕНИЯ

value objects

- ошибка валидации – не негативный сценарий бизнес логики
 - некорректный ввод
 - ошибка разработчика
- $\wedge$ оба по определению – исключительная ситуация

кроме валидаций

value objects

```
public record UserInfo(long UserId, long PostCount);

public UserInfo GetUserInfo(long userId)
{
    long postCount = ...;

    return new UserInfo(postCount, userId);
}
```


кроме валидаций

value objects

```
public readonly record struct UserId(long Value);

public record UserInfo(UserId UserId, long PostCount);

public UserInfo GetUserInfo(UserId userId)
{
    long postCount = ...;

    return new UserInfo(postCount, userId);
}
```

операторы

value objects

```
public void Process(Acceleration acceleration, TimeSpan time)
{
    var result = acceleration * time;
}
```

операторы

value objects

```
public void Process(Acceleration acceleration, TimeSpan time)
{
    var result = acceleration * time * time;
}
```


операторы

value objects

```
public void Process(Acceleration acceleration, TimeSpan time, Length segmentLength)
{
    var result = acceleration * time * time / segmentLength;
}
```

операторы

value objects

- операторы должны быть простыми
 - это верно не только для VO
 - операторы не должны скрывать преобразования типов
 - можно переопределять операторы, если параметры и возвращаемые значения – одного типа
 - можно переопределять операторы сравнения
 - $\wedge$ всё это в рамках разумного

преобразования

value objects

- для преобразований между типами – используйте методы
 - мы хотим абстрагироваться от деталей расчёта значений
 - но мы не хотим обфусцировать тот факт что расчет/преобразование имеет место

преобразования

value objects

```
public record Acceleration(double Value);  
  
public record Velocity(double Value);
```

преобразования

value objects

```
public record Acceleration(double Value);

public record Velocity(double Value)
{
    public static Velocity From(
        Acceleration acceleration,
        TimeSpan duration)
    {
        return new(acceleration.Value * duration.TotalSeconds);
    }
}
```


что делать *value object*'ом?

value objects

- VO используется для признания доменного примитива частью системы
- доменный примитив $\neq$ атрибут сущности
 - VO представляет величину в целом, а не величину в контексте сущности

что делать *value object*'ом?

value objects

```
public class MaterialDot
{
    public double AppliedForce { get; set; }

    public double MaxAppliedForce { get; set; }
}
```

что делать value object'ом?

value objects



```
public class MaterialDot
{
    public Force AppliedForce { get; set; }

    public MaxForce MaxAppliedForce { get; set; }
}
```



```
public class MaterialDot
{
    public Force AppliedForce { get; set; }

    public Force MaxAppliedForce { get; set; }
}
```


проектирование объектной модели структура ФС

проблематика

структура ФС

- структурирование файлов – важно
 - проще ориентироваться по коду
 - проще делать ревью

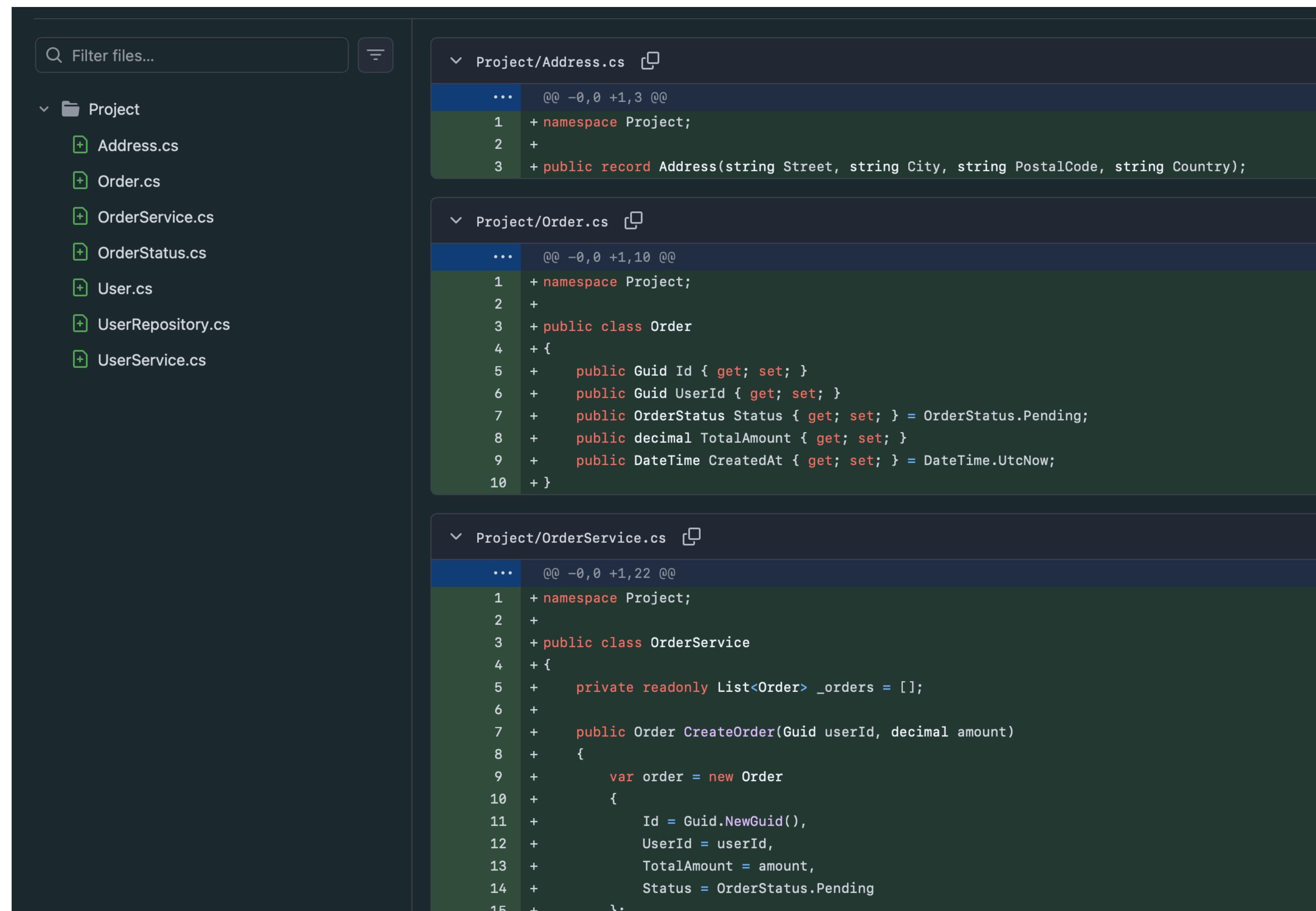
“СТИЛЬ вайбкодера”

структура ФС

- ▼ C# Project
 - >  Dependencies
 - C# Address.cs
 - C# Order.cs
 - C# OrderService.cs
 - C# OrderStatus.cs
 - C# User.cs
 - C# UserRepository.cs
 - C# UserService.cs

“СТИЛЬ вайбкодера”

структура ФС



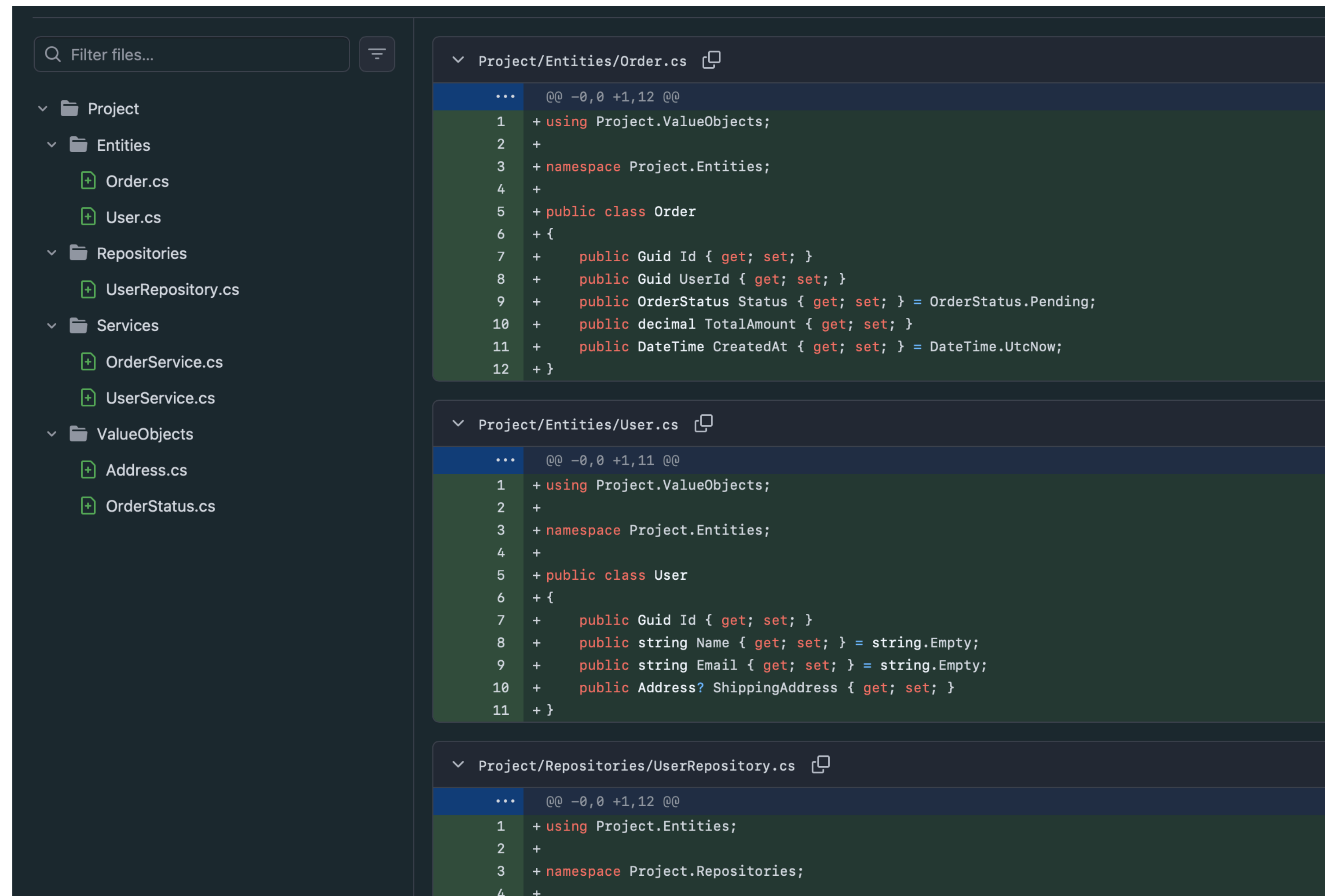
инфраструктурная группировка

структура ФС

- ▼  Project
 - >  Dependencies
 - ▼  Entities
 -  Order.cs
 -  User.cs
 - ▼  Repositories
 -  UserRepository.cs
 - ▼  Services
 -  OrderService.cs
 -  UserService.cs
 - ▼  ValueObjects
 -  Address.cs
 -  OrderStatus.cs

инфраструктурная группировка

структура ФС



доменная группировка

структура ФС

- ▼  Project
 - >  Dependencies
 - ▼  Orders
 -  Order.cs
 -  OrderService.cs
 -  OrderStatus.cs
 - ▼  Users
 - ▼  ValueObjects
 -  Address.cs
 -  UserId.cs
 -  User.cs
 -  UserRepository.cs
 -  UserService.cs

доменная группировка

структура ФС

